

GE_I Technical Design Document

로컬 LLM 장·단기 기억 시스템 설계

최고 수준의 충돌 검출(Conflict Detection)과
완전 회수(Exhaustive Recall)를 목표로 하는 기억 아키텍처

Project: Local AI Memory Stack (Personal R&D)

Target Runtime: OpenAI-compatible LLM server @ localhost:8080

Author: Scott Kim (김성수) · Reviewed by: Claude

Date: 2026-06-16 · Classification: Internal / Personal

Document History

Version	Date	Author	Description
1.0	2026-06-16	Claude / Scott	초판. 조사한 전 연구 내용 및 설계 통합.

Table of Contents

Document History	2
Table of Contents	3
1. Overview.....	5
1.1 목적과 설계 철학	5
1.2 런타임 제약 (Runtime Constraints)	5
1.3 문서 범위	6
2. 연구 기반 (Research Foundation)	6
2.1 기반 시스템 (Foundational Systems)	6
2.1.1 MemGPT / Letta — "LLM as OS" 계층적 가상 메모리 (2023.10)	6
2.1.2 Generative Agents (Stanford) — Memory Stream + Reflection (2023.04).....	7
2.2 프로덕션·벡터 기반 메모리	7
2.2.1 Mem0 / Mem0 ^g — 추출·통합 메모리 (2025, ECAI 2025)	8
2.2.2 A-MEM — Zettelkasten 기반 자기진화 메모리 (NeurIPS 2025).....	8
2.3 그래프·시간 기반 메모리	8
2.3.1 HippoRAG / HippoRAG 2 — 해마 인덱싱 + Personalized PageRank (NeurIPS'24 / ICML'25).....	8
2.3.2 Zep / Graphiti — Bi-temporal 지식그래프 메모리 (2025.01)	9
2.4 강화학습·학습형 메모리	9
2.4.1 Memory-R1 — RL 기반 메모리 관리 (2025.08).....	9
2.4.2 DeltaMem / LightMem (2025~2026)	10
2.5 망각·통합 메커니즘	10
2.5.1 MemoryBank — Ebbinghaus 망각 곡선 (AAAI 2024)	10
2.5.2 FadeMem / 지능형 망각 (2026).....	10
2.6 인지·계층 메모리 (수렴하는 설계 패턴).....	10
3. 충돌 검출: 유명 시스템의 실패와 처방	12
3.1 FactConsolidation 벤치마크 결과	12
3.2 처방 A — 결정론적 Freshness 규칙	12
3.3 처방 B — 비파괴 무효화 + Supersession Chain (Engram)	13
3.4 처방 C — 암묵적 충돌과 연쇄 무효화 (STALE).....	13
4. 완전 회수: 단 하나도 빠지지 않게	15
4.1 단일 신호의 필연적 누락	15
4.2 검증된 처방 — 하이브리드 검색 융합 (RRF).....	15
4.3 그래프 순회로 멀티홉 회수	15
4.4 누락과 환각을 가르는 Abstention 게이트	16
5. 시스템 아키텍처 (본 설계)	17

5.1 전체 구성도 (논리 계층).....	17
5.2 원본 자료 저장소 (Source Artifact Store).....	17
5.2.1 설계 원칙.....	17
5.2.2 데이터 모델 — Artifact 계층.....	18
5.2.3 모든 검색 경로 → 원본 수렴.....	19
5.3 데이터 모델 (Bi-temporal).....	19
5.3.1 Episode (대화 원문, 무손실).....	19
5.3.2 Fact (atomic SPO triple, bi-temporal).....	20
5.4 쓰기 경로 (Dual-Process).....	20
5.4.1 빠른 경로 — critical path에 LLM 없음.....	20
5.4.2 비동기 통합 — LLM @ :8080.....	21
5.5 충돌 해결 알고리즘 (핵심).....	21
5.6 읽기 경로 (Recall-First).....	22
6. 인터페이스 및 운영.....	24
6.1 LLM 호출 (OpenAI 호환, :8080).....	24
6.2 메모리 서비스 API (내부).....	24
6.3 유지보수 정책.....	25
7. 평가 (Evaluation).....	26
7.1 완전 회수 측정.....	26
7.2 충돌 검출 측정.....	26
7.3 효율.....	26
8. 구현 로드맵.....	27
8.1 즉시 시작 권고.....	27
Appendix A. 조사 연구 목록 (Reference Map).....	29

1. Overview

1.1 목적과 설계 철학

본 문서는 로컬 LLM 환경에서 동작하는 장·단기 기억 시스템의 설계를 기술한다. 이 설계의 두 가지 최우선 목표는 다음과 같으며, 이는 일반적인 "대화 기억" 시스템의 요구와 명확히 구분된다.

1. **충돌 검출(Conflict Detection)** — 조금이라도 관련 있는 내용이 존재할 때, 그들 사이에 내용 충돌이 없는지 빠짐없이 찾아내는 것.
2. **완전 회수(Exhaustive Recall)** — 특정 내용에 대해 검색하였을 때, 연관된 정보가 단 한 개도 누락되지 않도록 하는 것.
3. **원본 완전 회수(Artifact-level Recall)** — 위 회수 시, 그 기억과 연관된 원본 자료(대화 원문·스크린샷·파일)까지 어떤 검색 경로로든 빠짐없이 함께 회수하는 것.

이 두 목표는 메모리 연구 분야에서 가장 어렵고 가장 덜 풀린 문제다. 본 설계의 핵심 통찰은, 널리 알려진 상용·오픈소스 시스템(Mem0, Zep 등)이 일반 대화 벤치마크에서는 뛰어나지만 바로 이 두 축(특히 충돌 해결)에서는 처참하게 실패한다는 최신(2026년) 벤치마크 결과에 기반한다. 따라서 본 설계는 일반 권고와는 다른, 두 목표에 특화된 아키텍처를 채택한다.

핵심 원칙 (NOTE)

정확도(end-to-end accuracy)는 retrieval recall을 절대 초과할 수 없다. 올바른 정보가 검색되지 않으면 어떤 LLM도 정답을 낼 수 없다. $\text{recall_any}@k = 0.71$ 인 시스템은 LLM이 아무리 뛰어나도 최대 71% 정확도에 묶인다. 따라서 Scott의 용도에서 최우선 측정 지표는 답변 정확도가 아니라 retrieval recall이다.

1.2 런타임 제약 (Runtime Constraints)

항목	값 / 제약
LLM 추론 서버	OpenAI-compatible API, http://localhost:8080 (로컬 LLM)
호스트 하드웨어	Apple M4 Max MacBook (128 GB unified memory)
추론 스택	MLX 기반 로컬 모델 (function-calling 가능 모델 권장)
임베딩 모델	BGE-M3 (이미 보유, dense + sparse 동시 지원)
벡터 / 메타 저장소	Qdrant (이미 보유)

항목	값 / 제약
원본 자료 저장소	신규: content-addressable blob store (로컬 파일 또는 객체스토어) + 메타 DB
오케스트레이션	n8n 미사용 — 자체 Python 서비스로 비동기 처리 구현
전문 검색(FTS)	신규 도입 필요 (BM25 / SQLite FTS5 또는 Tantivy)
OCR / 텍스트 추출	신규: 이미지·스캔 PDF OCR, 문서 본문 추출 (로컬)

1.3 문서 범위

본 문서는 (a) 충돌 검출·완전 회수를 위해 조사된 모든 연구 근거, (b) 연구자들이 특별히 주의 깊게 관찰하고 조율한 설계 디테일, (c) 위 제약을 만족하는 구체적 시스템 아키텍처, (d) 데이터 모델·쓰기/읽기 경로·충돌 해결 알고리즘, (e) 원본 자료(대화 원문·스크린샷·파일) 저장 및 모든 검색 경로로부터의 완전 역추적, (f) API 설계와 평가 방법을 포함한다.

2. 연구 기반 (Research Foundation)

본 장은 설계의 근거가 된 모든 조사 내용을 빠짐없이 정리한다. 각 시스템·연구에 대해 핵심 메커니즘과, 특히 연구자들이 세심하게 조율한 부분(tuning points)을 명시한다.

2.1 기반 시스템 (Foundational Systems)

2.1.1 MemGPT / Letta — "LLM as OS" 계층적 가상 메모리 (2023.10)

분야의 출발점이자 가장 영향력 있는 기반 연구. OS의 가상 메모리(virtual memory)에서 영감을 받아, 물리 메모리(컨텍스트 윈도우)와 디스크 사이를 페이징하는 환상을 LLM에 제공한다. 3계층 구조가 핵심이다.

계층	OS 비유	역할
Main context	RAM	활성 윈도우: 시스템 프롬프트, 최근 메시지, 현재 관련 레코드
Recall storage	Disk	과거 모든 메시지의 검색 가능 DB
Archival storage	Cold storage	문서·장기 지식의 벡터 인덱스 저장소

주의 깊게 조율한 부분: LLM이 함수 호출([archival_memory_search](#), [core_memory_append](#) 등)을 통해

스스로 메모리를 관리한다. 인터럽트(interrupt) 메커니즘이 각 사용자 메시지·타이머 이벤트마다 제어권을 에이전트에 넘겨, 응답 전 여러 내부 메모리 연산을 수행하게 한다. 현재는 오픈소스 프레임워크 Letta로 진화했다.

2.1.2 Generative Agents (Stanford) — Memory Stream + Reflection (2023.04)

장기 메모리 검색의 사실상 표준 공식을 정립했다. 아키텍처는 세 요소로 구성된다: (1) 경험을 자연어로 기록하는 memory stream, (2) 메모리를 상위 추론으로 종합하는 reflection, (3) 행동 계획으로 변환하는 planning.

검색 스코어 공식 (이후 MemGPT, Mem0, LangGraph 등 거의 모든 시스템이 채택):

$\text{score} = \alpha_{\text{recency}} \cdot \text{recency} + \alpha_{\text{importance}} \cdot \text{importance} + \alpha_{\text{relevance}} \cdot \text{relevance}$ # Stanford 구현: 모든 $\alpha = 1$ (동일 가중)		
성분	계산 방식	연구자가 조율한 디테일
recency	마지막 접근 이후 시간의 지수 감소	decay factor = 0.995 (sandbox 게임 시간 기준)
importance	LLM에게 1~10 정수 점수를 직접 질의	"1=일상(양치) ~ 10=중대사(이별, 합격)" 프롬프트로 직접 평가가 효과적
relevance	쿼리-메모리 임베딩 간 코사인 유사도	세 성분을 [0,1] min-max 정규화 후 합산

Reflection은 최근 이벤트들의 importance 점수 합이 임계값을 넘으면 주기적으로 트리거되어, 원시 관찰을 추상적 상위 생각으로 종합한다(reflection tree). 이 종합된 메타-관찰은 다시 memory stream에 저장되어 이후 검색 대상이 된다.

주의 깊게 조율한 실패 모드 (CAUTION)

(1) Forgotten reflections: reflection을 recency만으로 스코어링하면 사라진다 → importance 스코어링이 필요하나 importance 인플레이션을 통제해야 함. (2) Context pollution: 검색 메모리가 너무 많으면 현재 질의를 압도함 → top-k는 3~5가 실용 범위, 10 초과는 거의 도움 안 됨. (3) Cross-agent leakage: 멀티 에이전트에서 memory stream은 단일 에이전트 소유여야 함.

2.2 프로덕션·벡터 기반 메모리

2.2.1 Mem0 / Mem0⁹ — 추출·통합 메모리 (2025, ECAI 2025)

현재 가장 널리 쓰이는 프로덕션 메모리 레이어. 대화에서 핵심 정보를 동적으로 추출(extraction)·통합·검색하는 2단계 아키텍처이며, 그래프 변형(Mem0⁹)은 관계 구조를 추가로 포착한다.

지표 (LOCOMO)	결과	비고
LLM-as-Judge 정확도	OpenAI 메모리 대비 +26% 상대 향상	66.9% vs 52.9%
p95 지연	91% 감소 (1.44s vs 17.12s)	선택적 검색 효과
쿼리당 토큰	7,000 미만	full-context 25,000+ 대비

한계(본 설계에 중요): 후속 분석(MemORAI, 2026)에 따르면 Mem0⁹의 그래프는 노드가 표면 이름만 저장하는 "shallow semantics"이고, synonym 엣지가 개념적 의미가 아닌 이름 유사도로 정의된다. 또한 뒤에서 보듯 충돌 해결 벤치마크에서 18%로 낮다.

2.2.2 A-MEM — Zettelkasten 기반 자기진화 메모리 (NeurIPS 2025)

고정 스키마를 버린 동적 메모리 조직화. Zettelkasten 방법의 원리를 따라 동적 인덱싱·링킹으로 상호 연결된 지식 네트워크를 만든다. 새 메모리가 추가되면 맥락 설명·키워드·태그를 포함한 구조화된 atomic note를 생성하고, 과거 메모리를 분석해 의미 있는 연결을 형성한다.

핵심 차별점 — 메모리 진화(memory evolution): 새 메모리가 통합되면서 기존 과거 메모리의 맥락 표현과 속성 업데이트를 트리거하여, 메모리 네트워크가 지속적으로 이해를 정제한다. 6개 foundation model에서 SOTA 대비 우수. 오픈소스 구현은 ChromaDB 기반(Qdrant로 교체 가능).

2.3 그래프·시간 기반 메모리

2.3.1 HippoRAG / HippoRAG 2 — 해마 인덱싱 + Personalized PageRank (NeurIPS'24 / ICML'25)

신경생물학(해마 인덱싱 이론) 기반의 지식 통합형 장기 메모리. 세 구성요소가 인간 장기기억의 상호 작용을 모사한다: 인공 신피질(LLM), 부해마 영역(PHR 인코더), 인공 해마(open KG).

단계	동작
오프라인 인덱싱	LLM이 구절을 KG 트리플로 변환 → 해마 인덱스에 적재. PHR이 동의어(synonymy)를 감지해 정보를 상호연결
온라인 검색	LLM이 쿼리의 named entity 추출 → PHR이 해마 인덱스에 연결 → KG 위에서 Personalized PageRank(PPR) 실행

성능: 멀티홉 QA에서 SOTA 대비 최대 20% 향상, 단일 단계 검색으로 반복 검색(IRCoT)과 동등하거나 우수하면서 10~20배 저렴, 6~13배 빠름. HippoRAG 2는 dense-sparse 결합 KG에 PPR을 적용.

2.3.2 Zep / Graphiti — Bi-temporal 지식그래프 메모리 (2025.01)

시간 추론에 특화된 프로덕션 메모리. 핵심 엔진 Graphiti는 동적·시간 인식 지식그래프로, 비정형 대화 데이터와 정형 비즈니스 데이터를 종합하면서 과거 관계를 보존한다. 인간 기억의 episodic/semantic 구분을 반영하는 세 서브그래프로 구성된다.

서브그래프	역할
Episode subgraph	에피소드 메모리: 각 노드 = 원본 이벤트/메시지 + 원본 타임스탬프
Semantic entity subgraph	추출된 엔티티·사실(bi-temporal 엣지 유효성 포함)
Community subgraph	label propagation으로 도출한 클러스터 수준 요약

Bi-temporal 모델 — 본 설계의 핵심 차용 (IMPORTANT)

Graphiti는 이벤트가 발생한 시점(t_{valid})과 수집된 시점(ingestion)을 모두 추적하며, 모든 엣지에 유효 구간 (t_{valid} , t_{invalid})을 명시한다. 새 지식이 기존과 충돌하면 시간 메타데이터로 오래된 정보를 폐기하지 않고 업데이트·무효화하여 역사적 정확성을 보존한다.

검색: 코사인 유사도 + BM25 전문검색 + breadth-first 그래프 순회를 조합하고, RRF·MMR·교차 인코더 reranking을 적용한 뒤, 사실을 유효 시간 범위와 함께 포맷하는 컨텍스트 구성기를 둔다. 임베딩·reranking에 BGE-M3 사용(Scott의 스택과 동일).

성능: DMR 벤치마크에서 MemGPT 능가(94.8% vs 93.4%), LongMemEval에서 정확도 최대 18.5% 향상 + 응답 지연 90% 감소.

2.4 강화학습·학습형 메모리

2.4.1 Memory-R1 — RL 기반 메모리 관리 (2025.08)

휴리스틱을 RL로 대체한 전환점. 두 전문 에이전트로 외부 메모리를 능동 관리한다.

- Memory Manager:** 구조화된 연산 {ADD, UPDATE, DELETE, NOOP}을 학습. 각 대화 턴에서 추출된 정보에 대해 적절한 연산과 갱신 내용을 결정.
- Answer Agent:** 관련 항목을 사전 선택(pre-select)하고 그 위에서 추론.

두 에이전트 모두 결과 기반 RL(PPO·GRPO)로 미세조정. 놀라운 데이터 효율성: 단 152개 QA 쌍으로 강력 베이스라인 능가, 세 벤치마크(LoCoMo, MSC, LongMemEval)·여러 규모(3B~14B)에서 일반

화. M4 Max에서 실험 가능하나 RL 파이프라인 구축 비용이 있어 고급 옵션.

2.4.2 DeltaMem / LightMem (2025~2026)

- **LightMem**: 인간 영감 메모리 계층으로 **메모리 구축을 온라인 추론에서 분리(decouple)**하여 효율 개선. — 로컬 환경에서 추론 연산을 아끼는 핵심 패턴.
- **DeltaMem**: RL 로 메모리 업데이트를 최적화하며 세션 간 시간적 진화를 명시적으로 모델링.

2.5 망각·통합 메커니즘

2.5.1 MemoryBank — Ebbinghaus 망각 곡선 (AAAI 2024)

"잊는 것"을 명시적으로 모델링한 선구 연구. Ebbinghaus 망각 곡선 이론에서 영감받은 갱신 메커니즘으로, 경과 시간과 메모리의 상대적 중요도에 따라 AI가 잊고 강화(forget & reinforce)하도록 한다. 페르소나 메모리를 atomic CRUD 연산으로 유지되는 구조화된 저장소로 형식화.

2.5.2 FadeMem / 지능형 망각 (2026)

생물학 영감 망각으로 retention과 효율을 균형. 핵심 사실 82.1% 보존하면서 저장소 55%만 사용. 중요 메모리는 baseline 대비 3~5배 느린 감쇠. 저접근 메모리 23%가 접근 패턴에 따라 장기 계층으로 승격. 전략으로는 TTL 계층·만료일을 사용하되 — ****불변 사실(예: 심각한 알레르기)에는 무한 TTL을 부여해 절대 감쇠하지 않게 함****.

2.6 인지·계층 메모리 (수렴하는 설계 패턴)

단일 논문이 아닌, 인지과학 모델로 수렴하는 설계 패턴. CoALA(Cognitive Architectures for Language Agents)는 working·episodic·semantic·procedural 저장소가 중앙 실행기(LLM)를 통해 상호 작용하는 청사진을 제안하며 Baddeley 모델을 직접 반영한다.

계층	정의	전형적 흐름
Working	현재 추론을 위한 활성 컨텍스트 (인지심리학상 3~5 청크)	Semantic→Working: 추론 전 관련 사실 검색·주입(RAG 패턴)
Episodic	시간 인덱스된 이벤트 시퀀스 / 세션 요약	Working→Episodic: 발생한 이벤트를 영속화
Semantic	엔티티·사실의 추상화된 장기 지식	벡터/KG로 저장, 오래된 정보의 주 저장소
Procedural	모델 가중치·시스템 프롬프트·도구 레지스트리	천천히 변하며 명시적 갱신(배포·파인튜닝) 필요

계층	정의	전형적 흐름
계층 메모리의 아킬레스건 — 오케스트레이션 (CAUTION) 잘못된 것을 페이지징해 넣으면 토큰을 낭비하고, 너무 공격적으로 보관(archive)하면 "memory blindness"가 생긴다 — 에이전트가 중요한 사실이 cold storage에 존재한다는 것 자체를 모르게 됨. 그리고 오케스트레이션 실패는 조용히(silently) 일어난다. 단기 버퍼는 4페이지(STIM)로 엄격히 제한하는 것이 인지과학 근거(작업기억 3~5 청크)와 부합한다.		

3. 충돌 검출: 유명 시스템의 실패와 처방

Scott이 강조한 첫 번째 목표(충돌 검출)에 대해, 2026년 발표된 전용 벤치마크가 충격적인 사실을 드러냈다. 이 장은 그 근거와 본 설계가 채택하는 처방을 기술한다.

3.1 FactConsolidation 벤치마크 결과

MemoryAgentBench(MAB; Hu, Wang, McAuley, 2026)의 FactConsolidation 태스크는 "Selective Forgetting" 역량 하에서 충돌 해결을 측정한다. 사실에 번호가 매겨지고 반사실(counterfactual)이 더 높은 serial로 등장하며, 에이전트에게 "더 새로운 사실은 더 큰 serial 번호를 가진다"는 규칙을 명시적으로 알려준다. 규칙을 그대로 알려줬는데도 모든 발표된 시스템이 크게 미달했다.

시스템	FC-SH (single-hop) 정확도	설계 특성
HippoRAG-v2	54.0%	KG + PPR (최고점이나 절반 수준)
BM25	48.0%	순수 어휘 검색
Mem0 / Contriever	18.0%	추출·통합 메모리
Zep / Graphiti	7.0%	bi-temporal KG (충돌 처리용 설계임에도 최저)

결정적 통찰 (IMPORTANT)

Zep은 supersession 처리를 위해 설계됐는데도 충돌 벤치마크에서 7%로 꼴찌였다. 그럴듯한 원인은, Zep이 ingestion 시점에 LLM을 사용해 새 사실이 기존과 모순되는지 판단(resolve_edge 프롬프트)하기 때문이다. 즉 temporal-KG 인프라조차도 해결 결정을 LLM 프롬프트가 내린다면 충돌 해결을 자동으로 풀어주지 않는다.

→ 설계 원칙 #1: 충돌 해결 결정을 LLM의 판단에 맡기지 마라. LLM은 자신의 저장된 지식과 모순되는 컨텍스트를 자주 무시하며, 외부 컨텍스트 수용 여부가 기존 믿음과의 일치 여부에 좌우된다.

3.2 처방 A — 결정론적 Freshness 규칙

"Don't Ask the LLM to Track Freshness: A Deterministic Recipe for Memory Conflict Resolution" (arXiv 2606.01435, 2026) 논문의 처방. 충돌 해결을 LLM이 아니라 결정론적 메타데이터 규칙으로 처리한다. 같은 (subject, predicate)에 대해 값이 충돌하면, 최신 timestamp/serial이 자동으로 이긴다. LLM 호출 없음. 이것이 위 표의 LLM-판단 실패 모드를 제거한다.

3.3 처방 B — 비파괴 무효화 + Supersession Chain (Engram)

"Less Context, More Accuracy: A Bi-Temporal Memory Engine" (Engram, arXiv 2606.09900, 2026).
본 설계가 가장 정확히 차용하는 시스템이다. 이중 처리(dual-process) + bi-temporal 데이터 모델 위에서 동작한다.

6. **빠른 쓰기 경로(fast write path)**: LLM 없이 무손실 episode 를 즉시 추가(critical path 에 LLM 없음).
7. **비동기 경로(asynchronous path)**: atomic (subject, predicate, object) 사실을 추출 → bi-temporal KG 구축 → **사실당 LLM 호출 없이** 모순 해결.
8. **비파괴(non-destructive)**: 모순된 사실은 무효화(invalidate)하되 **절대 삭제하지 않음** → 모든 사실이 provenance 와 supersession chain 을 유지.

왜 '삭제하지 않고 무효화'가 결정적인가 (NOTE)

Scott의 요구는 충돌을 "찾아내는" 것이다. 만약 덮어쓰기(overwrite)하면 충돌이 있었다는 사실 자체가 사라진다. 무효화 + supersedes chain은 "A였다가 B로 바뀌었고, 그 근거(provenance)는 이것"이라는 전체 이력을 보존한다 — 엔지니어링 의사결정 추적(IP 일정·버스 구조·담당자 변경 등)에 정확히 필요한 구조다.

검증된 효과: bi-temporal 모델링은 knowledge-update(무효화를 통한 최신 우선)에서 87.5%, temporal-reasoning(날짜 스탬프 + as-of 필터링)에서 81.1% 도달. 전체 LongMemEval_S 500문항에서 lean 구성(~9.6k 토큰 검색 슬라이스)이 full-context(73.2%) 대비 83.6%로 +10.4점, 약 8배 적은 토큰, 0/500 오류.

3.4 처방 C — 암묵적 충돌과 연쇄 무효화 (STALE)

가장 까다로운 케이스. STALE 논문(arXiv 2605.06527, 2026)이 두 유형을 정의한다.

- **Type I (co-referential) 충돌**: 두 관찰이 같은 기저 속성을 갱신하면서 표면적으로는 양립 가능.
예) "시애틀에 산다"가, "포틀랜드에서 새 임대 계약·공과금 설정"이라는 이후 진술에 의해 — "시애틀을 떠났다"는 명시 없이도 — 암묵적으로 무효화됨.
- **Cascading Invalidation(연쇄 무효화)**: 한 속성의 갱신이 구조적으로 관련된 속성들을 무효화.

Scott 도메인 예시 (TIP)

"IP X는 5/30까지 수령"이라는 사실이 있는데 나중에 "버스 구조 CCI-550 미사용 확정"이 들어오면, CCI 관련 대역폭 계산 사실들이 연쇄적으로 stale해진다. 표면에 모순 단어는 없다. 이 케이스

는 순수 결정론으로 안 잡히므로, same-(subject,predicate) 결정론 규칙 + 그래프 상 연결 엔티티 재검토를 병행해야 한다 (본 설계 5.4절).

RAG의 근본 한계도 명시한다: 표준 RAG는 의미 유사도를 시간 상태 해소(temporal state resolution)보다 우선하기 때문에, 나중의 부상(injury) 관찰이 자전거 추천을 부적절하게 만들어야 함에도 오래된 cycling 메모리를 검색할 수 있다. 즉 메모리 시스템은 대화 스니펫을 캐싱하는 것이 아니라 진화하는 잠재 사용자 상태(latent state)의 일관된 표현을 구축해야 한다.

4. 완전 회수: 단 하나도 빠지지 않게

Scott의 두 번째 목표. 1.1절의 핵심 원칙(정확도 $\leq$ recall)에 따라, 본 설계는 생성 품질보다 retrieval recall을 절대 우선한다.

4.1 단일 신호의 필연적 누락

벡터 유사도만으로는 반드시 놓친다. 의미는 다르지만 같은 용어를 쓰는 항목, 표면형은 다르지만 관련된 항목이 빠진다. 특히 Scott처럼 정확한 기술 용어(IP 이름, 신호명, 프로젝트 코드네임 — slice_master, DQSDLY, CCI-550 등)가 중요한 도메인에서는 벡터 임베딩이 이를 흐릿하게 만들지만 어휘 검색(BM25/FTS)은 정확히 잡는다.

4.2 검증된 처방 — 하이브리드 검색 융합 (RRF)

dense + lexical + graph + recency를 Reciprocal Rank Fusion으로 융합한다. 한 로컬 구현(Aingram)은 이 방식으로 LongMemEval에서 recall_any@10 = 0.955를 달성했다 — 즉 95.5%의 경우에 올바른 자료를 LLM 앞에 놓는다.

채널	강점	본 설계 구현
Dense semantic	의미 유사도	BGE-M3 dense (보유)
Lexical (BM25/FTS)	정확한 용어·고유명사	신규: SQLite FTS5 또는 Tantivy
Graph traversal	엔티티 관계·멀티홉 연결	Qdrant 메타 + 관계 엣지 n-hop
Recency / salience	최신성·중요도	타임스탬프 + importance 점수

Engram의 read 경로: 멀티홉 질문은 sub-query로 분해 → 4개 상보 채널 병렬 검색 → RRF 융합 → (선택적) 교차 인코더 reranking → as-of 시점 필터 → abstention gate → 중복 제거된 provenance 태그 컨텍스트 조립. 결정적으로, 조립 컨텍스트는 hybrid다 — 충돌 해결된 bi-temporal 사실 + 가장 관련된 원본 청크를 함께 담는다 (사실만으로는 recall이 떨어지므로).

4.3 그래프 순회로 멀티홉 회수

흩어진 관련 정보를 빠짐없이 모으려면 그래프 순회가 필요하다. entity·temporal·semantic·causal 링크가 함께 메모리 그래프 전반의 멀티홉 발견을 지원하여, 표면형만이 아니라 정체성·시간·의미·설명으로 관련된 정보를 surface한다(Hindsight/TEMPR). HopRAG는 그래프 구조의 논리적 관계에 기반해 간접적으로 관련된 구절의 이웃을 BFS로 탐색하여 관련 구절로 hop한다.

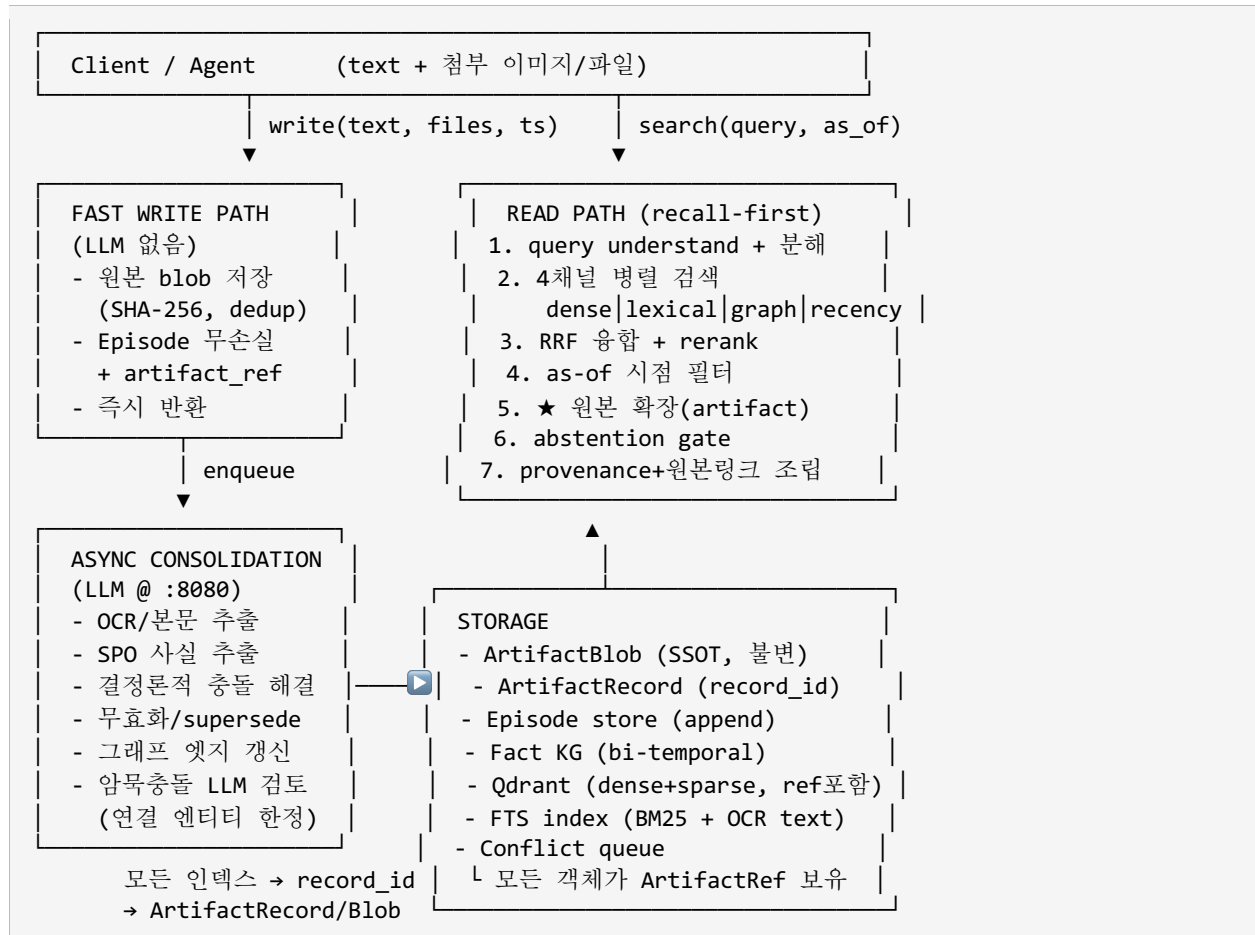
4.4 누락과 환각을 가르는 **Abstention** 게이트

완전 회수의 짝은 "없으면 없다고 말하기"다. 빠뜨린 채 그럴듯하게 지어내는 것이 최악이다. 시스템은 정보가 없을 때 추측·환각하지 않고 올바르게 답변을 거부(abstain)해야 한다. Scott의 용도에서는 "관련 정보 0개를 빠뜨림"과 "정보 없는데 지어냄"이 둘 다 치명적이므로 abstention gate가 필수다.

5. 시스템 아키텍처 (본 설계)

위 연구를 종합한, 꼼꼼함 최우선 아키텍처. n8n 없이 자체 Python 비동기 서비스로 구현하며, LLM은 localhost:8080의 OpenAI 호환 API로만 호출한다.

5.1 전체 구성도 (논리 계층)



5.2 원본 자료 저장소 (Source Artifact Store)

Scott의 완전 회수 요구를 자료 계층까지 확장한다. 핵심 원칙: 모든 검색 가능한 객체(Episode, Fact, 임베딩 청크, 그래프 노드/엣지)는 불변(immutable) 원본 레코드를 가리키는 안정적 링크를 보유하며, 어떤 검색 경로(RAG·graph·lexical·recency)로 도달하든 동일한 원본으로 수렴한다. 원본은 한 번만 저장하고 모든 곳에서 참조한다.

5.2.1 설계 원칙

9. **Content-addressable 저장**: 모든 원본(대화 원문·스크린샷·파일)은 SHA-256 콘텐츠 해시를 기본키로 저장 → 동일 파일 중복 제거(dedup), 무결성 검증, 안정적·영구적 주소 확보.
10. **불변(append-only)**: 원본 레코드는 절대 수정·삭제하지 않음 (§3.3 비파괴 원칙의 자료 계층 확장). 갱신은 새 버전 레코드 추가 + supersedes 링크.
11. **단일 진실 원본(SSOT)**: 원본 바이트는 ArtifactBlob 에 1 회 저장. Episode·Fact·청크·그래프 노드는 모두 ArtifactRef(가벼운 포인터)만 보유 → 어느 경로로 와도 같은 blob 으로 역추적.
12. **양방향 링크**: 원본→파생(이 원본에서 나온 Episode/Fact/청크)과 파생→원본을 모두 인덱싱 → "이 기억의 원본"과 "이 원본이 등장한 모든 기억" 양방향 회수.

5.2.2 데이터 모델 — Artifact 계층

```

ArtifactBlob {
    content_hash: str (SHA-256)    # ★ 기본키 = 콘텐츠 주소
    mime_type:    str              # text/plain, image/png, application/pdf ...
    byte_size:    int
    storage_uri:  str              # 로컬 blob 경로 또는 객체스토어 키
    created_at:   datetime
    kind:         {conversation, screenshot, file, document, audio}
}

ArtifactRecord {
    id:           UUID             # ★ DB record id (모든 링크가 가리키는 대상)
    blob_hash:    str -> ArtifactBlob.content_hash
    title:        str|null         # 파일명/캡션
    ocr_text:     str|null         # 이미지·스캔 PDF의 추출 텍스트(검색용)
    extracted_text: str|null       # 파일 본문 텍스트
    thumb_uri:    str|null         # 미리보기(이미지/PDF 첫 페이지)
    page_map:     [PageSpan]|null  # PDF 페이지↔텍스트 오프셋(정밀 인용)
    source_ctx:   str              # 어느 대화/세션에서 첨부됐는지
    t_event:      datetime
    t_ingest:     datetime
    supersedes:   UUID|null       # 파일 갱신 시 이전 버전
}

ArtifactRef {
    record_id:    UUID -> ArtifactRecord.id
    blob_hash:    str              # 무결성 재검증용
    span:         [start,end]|null # 원문/문서 내 정확한 구간
    page:         int|null         # PDF 페이지
    bbox:         [x,y,w,h]|null   # 이미지 내 영역(있으면)
    role:         {origin, mention, evidence}
}

```

연구자 조율 포인트 — 자료 회수 (IMPORTANT)

(1) ArtifactRef는 record_id + blob_hash를 함께 들고 있어, 링크가 가리키는 레코드가 갱신·이동돼

도 blob_hash로 원본 무결성을 재검증한다. (2) span/page/bbox로 "원본의 어느 부분"인지까지 정밀 회수 — 단순 파일 첨부가 아니라 인용 지점까지 복원. (3) 이미지·스캔 PDF는 OCR 텍스트를 ArtifactRecord에 함께 색인해, 텍스트 검색만으로도 이미지 원본이 회수된다(멀티모달 회수의 핵심). (4) content_hash 기반 dedup으로 같은 스크린샷이 여러 대화에 등장해도 단일 원본으로 수렴.

5.2.3 모든 검색 경로 → 원본 수렴

어떤 검색을 하더라도 결과 객체에서 ArtifactRef를 따라 원본으로 100% 역추적되도록, 각 인덱스가 record_id를 페이로드로 보유한다.

검색 경로	1차 히트 객체	원본 회수 방식
RAG (dense, BGE-M3)	임베딩 청크	청크 payload의 artifact_ref.record_id → ArtifactRecord → blob
Lexical (BM25/FTS)	텍스트 청크 / OCR 텍스트	동일 record_id 역추적. 이미지도 OCR 텍스트로 히트 시 원본 회수
Graph 순회	Fact / 엔티티 노드	Fact.provenance→Episode.artifact_refs, 노드의 ArtifactRef
Recency / salience	Episode	Episode.self_ref + artifact_refs
충돌 검토	Fact (active/invalidated)	supersession chain의 각 Fact가 자신의 근거 원본 링크 보유

불변식 (NOTE)

INVARIANT: 검색 결과로 반환되는 모든 객체는 비어있지 않은 ArtifactRef를 최소 1개 보유해야 한다(쓰기 시 강제). 이를 만족하지 못하는 파생 객체는 색인 자체를 거부 → "원본 없는 기억"을 구조적으로 차단. 평가 단계에서 artifact_recall@k(원본까지 회수했는가)를 별도 측정한다(§7).

5.3 데이터 모델 (Bi-temporal)

5.3.1 Episode (대화 원문, 무손실)

```
Episode {
  id:          UUID
  raw_text:    str          # 대화 원문(무손실)
  source:      str          # 출처 (세션/문서/턴)
  t_event:     datetime     # 발생 시점(유효시간축)
```

```

t_ingest:      datetime      # 수집 시점(트랜잭션측)
embedding:     vector(BGE-M3 dense)
sparse:        sparse_vector(BGE-M3) # 어휘 신호
artifact_refs: [ArtifactRef] # ★ 첨부 원본(이미지/파일) 링크 목록
self_ref:      ArtifactRef   # ★ 이 원문 자체의 불변 레코드 링크
}

```

5.3.2 Fact (atomic SPO triple, bi-temporal)

```

Fact {
  id:          UUID
  subject:     str          # 정규화된 엔티티
  predicate:   str          # 정규화된 관계/속성
  object:      str          # 값
  t_valid_from: datetime    # 유효 시작
  t_valid_to:  datetime|null # 유효 종료(무효화 시 기입)
  t_ingest:    datetime
  status:      {active, invalidated}
  supersedes:  UUID|null    # 이전 사실 포인터(체인)
  superseded_by: UUID|null
  provenance:  [Episode.id] # 근거 에피소드
  artifact_refs: [ArtifactRef] # ★ provenance 경유로 도달하는 원본 링크(역정규화 캐시)
  importance:  float        # 1~10 (LLM 1회 평가, 캐시)
  confidence:  float
}

```

연구자 조율 포인트 (IMPORTANT)

(1) status=invalidated 사실도 절대 삭제하지 않는다 → as-of 질의로 과거 시점 상태를 재구성 가능. (2) supersedes/superseded_by 양방향 포인터로 supersession chain을 완전 추적. (3) importance는 Generative Agents의 1~10 LLM 평가를 1회만 수행해 캐시(매번 호출 금지). (4) 불변 사실(예: 확정된 사양·계약 조항)은 t_valid_to=null + 무한 TTL로 감쇠에서 제외(FadeMem 원칙).

5.4 쓰기 경로 (Dual-Process)

5.4.1 빠른 경로 — critical path 에 LLM 없음

13. 첨부 원본 우선 저장: 대화 원문·스크린샷·파일을 ArtifactBlob(content-hash)로 즉시 저장 → ArtifactRecord 생성, record_id 발급. 동일 해시는 dedup.
14. 입력 텍스트를 Episode 로 즉시 append-only 저장하며, self_ref 및 첨부 artifact_refs 를 연결(무손실).
15. BGE-M3 로 dense+sparse 임베딩 생성 → Qdrant upsert. 각 청크 payload 에 artifact_ref(record_id+blob_hash+span)를 필수 포함. FTS 인덱스 색인.

16. 비동기 통합 큐에 enqueue 후 즉시 반환 (사용자 대기 없음 = LightMem 패턴).

5.4.2 비동기 통합 — LLM @ :8080

17. 이미지/파일 텍스트화: 스크린샷·스캔 PDF 는 OCR, 문서는 본문 추출 →

ArtifactRecord.ocr_text/extracted_text 채우고 별도 청크로 색인(이미지도 텍스트 검색으로 회수 가능).

18. LLM 에 SPO 추출 프롬프트 호출(배치) → atomic (subject, predicate, object) 사실 추출 + subject/predicate 정규화(동의어 사전 갱신). 추출된 Fact 에 근거 Episode 의 artifact_refs 를 역정규화 캐시.

19. 각 신규 사실에 대해 §5.5 결정론적 충돌 해결 수행.

20. 그래프 엣지(엔티티-엔티티, 사실-에피소드 provenance, 엔티티-아티팩트) 갱신.

21. 표면적으로 안 겹치는 암묵적 충돌 후보만 LLM 에 검토 요청(연결 엔티티 한정).

5.5 충돌 해결 알고리즘 (핵심)

LLM 판단을 최소화하고 결정론을 우선한다. 신규 사실 f_new 도착 시:

```
def resolve(f_new):
    # 1) 동일 (subject, predicate) 활성 사실 조회
    actives = kg.find_active(f_new.subject, f_new.predicate)

    for f_old in actives:
        if f_old.object == f_new.object:
            # 값 동일 → 강화(reinforce), 중복 누적 금지
            f_old.confidence += δ
            f_old.importance = max(f_old.importance, f_new.importance)
            f_old.provenance += f_new.provenance
            return REINFORCED
        else:
            # 값 충돌 → 결정론적 freshness: 최신 우선
            if f_new.t_valid_from >= f_old.t_valid_from:
                f_old.t_valid_to = f_new.t_valid_from
                f_old.status = 'invalidated' # 삭제 아님
                f_old.superseded_by = f_new.id
                f_new.supersedes = f_old.id
                kg.add(f_new); return SUPERSEDED
            else:
                # 더 오래된 사실 → 과거 유효구간으로 삽입
                kg.insert_historical(f_new); return BACKFILLED

    # 2) 활성 충돌 없음 → 신규 추가
    kg.add(f_new); return ADDED
```

암묵적/연쇄 충돌(§3.4) 처리 — 결정론으로 잡히지 않는 케이스만:

22. f_new 의 subject/object 와 그래프 상 n-hop 연결된 엔티티의 활성 사실들을 "재검토 후보"로 수집.
23. 후보가 임계치 이상이면 LLM 에 "이 갱신이 후보 사실들을 stale 하게 만드는가?"를 배치 질의 (Type I/cascading 탐지).
24. LLM 이 stale 판정한 사실은 자동 무효화하지 않고 **conflict queue 에 등재** → Scott 이 직접 검토·확정. 큐 항목은 양측 사실의 원본 ArtifactRef 를 함께 제시(원문·스크린샷을 바로 열람).

왜 자동 무효화하지 않고 큐에 넣는가 (CAUTION)

암묵적 충돌은 LLM 판단에 의존하므로(§3.1 실패 모드) 불안정하다. "조금이라도 관련 있을 때 충돌을 찾아낸다"는 Scott의 요구는 충돌을 자동 해소하라는 뜻이 아니라 빠짐없이 발견·제시하라는 뜻이다. 따라서 결정론으로 확실한 것만 자동 처리하고, 모호한 것은 사람이 검토하도록 conflict queue로 보낸다. 미해결 충돌은 항상 가시화한다(Engram CLI의 "show unresolved contradictions"와 동일 철학).

5.6 읽기 경로 (Recall-First)

25. **질의 이해·분해**: 멀티홉 질문은 LLM(@:8080)으로 sub-query 분해.
26. **4 채널 병렬 검색**: (a) BGE-M3 dense, (b) BM25/FTS 어휘(OCR 텍스트 포함), (c) 쿼리 엔티티로부터 n-hop 그래프 순회, (d) recency/saliency. 각 채널 top-N 을 넉넉히(recall@20~30) 확보.
27. **RRF 융합**: 네 순위 리스트를 Reciprocal Rank Fusion 으로 결합. 신호가 복수 일치하면 dual-hit 가중.
28. **Rerank**: 상위 T 개에 (선택적) 교차 인코더 reranking (BGE reranker).
29. **as-of 시점 필터**: 질의에 명시/암시된 시점 기준으로 t_valid 범위 필터링 → 과거 시점 질의 정확 지원.
30. ★ **원본 확장(artifact expansion)**: 각 히트의 ArtifactRef 를 따라 ArtifactRecord/Blob 을 회수. content_hash 로 dedup 하여 동일 원본 중복 제거. 텍스트 청크가 히트하면 그 청크가 속한 원본 파일/이미지 전체로 확장(부분→전체 회수).
31. **Abstention gate**: 증거가 임계 이하면 답변 거부(추측·환각 금지).
32. **컨텍스트 조립**: 충돌 해결된 활성 사실 + 원본 청크(hybrid) + provenance 태그 + **원본 자료 링크/썸네일/인용 span**, 토큰 예산 내 중복 제거. 멀티모달 모델이면 이미지 원본을 직접 첨부.

부분 히트 → 전체 원본 회수 (TIP)

RAG는 보통 청크 단위로 히트한다. 이 설계는 청크의 `artifact_ref`로 원본 레코드 전체를 회수하므로, "PDF 3페이지의 한 문장"이 검색되면 해당 PDF 원본 전체와 정확한 페이지·span을 함께 반환한다. 스크린샷이 OCR 텍스트로 히트하면 이미지 원본 자체를 반환한다. 즉 어떤 검색 경로든 연관 원본이 누락 없이 따라온다.

6. 인터페이스 및 운영

6.1 LLM 호출 (OpenAI 호환, :8080)

모든 LLM 추론은 단일 엔드포인트로 통일한다. SPO 추출·암묵충돌 검토·sub-query 분해·importance 평가에 사용.

```
POST http://localhost:8080/v1/chat/completions
Content-Type: application/json
{
  "model": "<local-model>",          # function-calling 가능 모델 권장
  "messages": [...],
  "temperature": 0,                  # 추출·판정은 결정성 위해 0
  "response_format": {"type": "json_object"} # 구조화 출력
}
```

연구자 조율 포인트 — 추출 결정성 (NOTE)

SPO 추출·충돌 판정은 temperature=0 + JSON 강제로 결정성을 높인다. 임베딩은 BGE-M3로 로컬 처리하여 :8080 LLM 부하와 분리(LightMem의 construction/inference 분리 원칙). 비동기 통합은 배치로 묶어 호출 횟수를 최소화(M4 Max 연산 절약).

6.2 메모리 서비스 API (내부)

엔드포인트	설명
POST /write	텍스트+첨부(이미지/파일)+타임스탬프 저장. 원본 blob 저장 후 즉시 반환, 통합은 비동기.
GET /search	query, as_of?, k? → recall-first 하이브리드 검색 + 원본 ArtifactRef/썸네일 포함 결과.
GET /artifact/{record_id}	원본 레코드 메타 + blob 다운로드 링크 + 인용 span/page.
GET /artifact/{record_id}/derived	이 원본에서 파생된 모든 Episode/Fact/청크 (원본→기억 역방향 회수).
GET /conflicts	미해결 충돌 큐 조회 (양측 원본 링크 포함, 사람 검토용).
POST /conflicts/{id}/resolve	충돌을 전략(supersede/keep-both/reject)으로 확정.
GET /fact/{id}	사실 상세 + provenance + supersession chain + 원본 ArtifactRef.

엔드포인트	설명
POST /reinforce/{id}	사실 강화(중요도·confidence 상승).
POST /consolidate	통합 1회 수동 실행(디버그/배치).

6.3 유지보수 정책

- **보수적 pruning**: Scott 의 "빠짐없이"가 우선이므로 무효화 사실도 검색 가능하게 유지. as-of 로 시점 조회. 물리 삭제는 하지 않음.
- **감쇠는 가시성에만 적용**: 저접근·저중요 사실은 기본 검색 순위에서 내려가되 제거되지 않음(MemoryBank/FadeMem). 불변 사실은 무한 TTL.
- **강화(reinforcement)**: 동일 값 재등장 시 중복 누적 대신 confidence 상승 — 저장소 lean 유지하면서 누락 방지.
- **원본 불변 보존**: ArtifactBlob/Record 는 절대 삭제하지 않음(append-only). 파일 갱신은 새 버전 + supersedes 링크. content-hash dedup 으로 용량은 억제하되 원본 회수는 영구 보장.

7. 평가 (Evaluation)

"측정하지 않으면 의미 없다." 본 설계는 답변 정확도가 아니라 두 목표를 직접 측정한다.

7.1 완전 회수 측정

- **recall_any@k** 를 1 차 지표로 추적 (정확도가 아님). 목표: $\text{recall_any}@10 \geq 0.95$ (Aingram 로컬 기준선).
- ★ **artifact_recall@k**: 검색 히트로부터 연관 원본(대화 원문·스크린샷·파일)이 누락 없이 회수되는 비율. 불변식(§5.2.3)에 따라 목표 = 1.00. 텍스트→원본, OCR→이미지, 청크→전체 파일 회수를 각각 검증.
- 채널별 ablation: dense 단독 vs +lexical vs +graph vs +recency — 각 채널이 회수하는 고유 항목 수를 측정해 누락원 식별.
- LongMemEval_S(500 문항, ~115k 토큰/문항) 회수 전용 평가 + 멀티모달 첨부 회수 자가 셋.

7.2 충돌 검출 측정

- **FactConsolidation 형식 자가 테스트**: 같은 (subject,predicate)에 모순 값을 시간차로 주입하고, 시스템이 최신을 고르고 과거를 supersession chain 으로 보존하는지 검증(single-hop / multi-hop).
- 암묵적 충돌(STALE Type I) 회귀 셋: 표면 비충돌이지만 상태를 갱신하는 케이스를 conflict queue 가 포착하는 비율.
- **비파괴 검증**: 무효화 후에도 과거 시점 as-of 질의가 옛 값을 정확히 복원하는지.

7.3 효율

- 쿼리당 토큰(목표 < 10k, Engram lean 기준), p95 검색 지연, :8080 LLM 호출 횟수(배치 효율).

8. 구현 로드맵

단계	내용	근거
Phase 1 (기반)	ArtifactBlob/Record(content-hash) 원본 저장 + Episode append-only + Qdrant dense/sparse(ref 포함) + FTS5 색인 + RRF 융합 읽기 경로 + 원본 확장. recall_any@k / artifact_recall@k 측정 하니스 구축.	§4.2, §5.2, §7.1 — 회수·원본 기반 먼저
Phase 2 (충돌)	비동기 SPO 추출 + OCR/본문 추출 + bi-temporal Fact KG + 결정론적 freshness 충돌 해결 + 비파괴 무효화/supersession.	§3.2~3.3, §5.5
Phase 3 (암묵충돌)	그래프 n-hop 재검토 + LLM 암묵충돌 검토(배치) + conflict queue(원본 링크 포함) + 사람 검토 API.	§3.4, §5.5
Phase 4 (고도화)	교차 인코더 rerank, importance/decay 가시성 정책, 멀티모달 컨텍스트 첨부, (선택) Memory-R1식 RL 관리 도입.	§2.1.2, §2.4, §2.5

8.1 즉시 시작 권고

가장 빠른 PoC: "Episode 무손실 저장 + BGE-M3(dense)/FTS(BM25)/그래프/recency 4채널 RRF 검색(회수) + 비동기 SPO 추출 + 결정론적 freshness 충돌 해결(비파괴)". 이 조합이 Scott의 두 목표(충돌 검출·완전 회수)를 연구상 가장 안정적으로 달성한다. RL(Memory-R1)이나 무거운 KG 커뮤니티 요약은 기본 파이프라인 안정화 후 도입한다.

최종 요약 (IMPORTANT)

충돌 검출 → LLM 판단 금지. 결정론적 (subject,predicate) freshness 규칙 + 비파괴 무효화 + supersession chain. 암묵·연쇄 충돌만 그래프 이웃 재검토로 보완하고, 모호한 것은 conflict queue로 사람이 확정. 미해결 충돌은 항상 가시화.

완전 회수 → 단일 신호 금지. dense+lexical+graph+recency 4채널 RRF 융합(특히 기술 용어엔 BM25 필수). 멀티홉 분해. recall_any@k를 최우선 지표로. abstention으로 누락과 환각을 구분.

원본 회수 → 모든 원본(대화 원문·스크린샷·파일)을 content-hash 불변 blob으로 1회 저장(SSOT). 모든 검색 가능한 객체가 ArtifactRef를 보유해, RAG·graph·lexical·recency 어떤 경로로 도달해도

동일 원본으로 100% 역추적(불변식). 이미지는 OCR 텍스트로도 회수되고, 청크 히트는 원본 전체+인용 span으로 확장. artifact_recall@k = 1.00을 목표로 강제·측정.

Appendix A. 조사 연구 목록 (Reference Map)

본 설계가 참조한 모든 연구를 영역별로 정리한다.

영역	연구 / 시스템	식별자 / 비교
기반	MemGPT / Letta	arXiv 2310.08560 (2023)
기반	Generative Agents (Stanford)	arXiv 2304.03442, UIST 2023
벡터/프로덕션	Mem0 / Mem0 ⁹	arXiv 2504.19413, ECAI 2025
벡터/자기진화	A-MEM (Zettelkasten)	arXiv 2502.12110, NeurIPS 2025
그래프	HippoRAG / HippoRAG 2	NeurIPS'24 (2405.14831) / ICML'25 (2502.14802)
시간/그래프	Zep / Graphiti	arXiv 2501.13956 (2025)
RL	Memory-R1	arXiv 2508.19828 (2025)
RL/효율	DeltaMem / LightMem	arXiv 2604.01560 / Fang et al. 2025
망각	MemoryBank (Ebbinghaus)	AAAI 2024
망각	FadeMem	arXiv 2601.18642 (2026)
인지/계층	CoALA / H-MEM / 3-Layer	Sumers 2024 / 2507.22925 / 2602.14038
충돌(핵심)	결정론적 Freshness Recipe	arXiv 2606.01435 (2026)
충돌(핵심)	Engram (bi-temporal engine)	arXiv 2606.09900 (2026)
충돌	STALE (암묵·연쇄 충돌)	arXiv 2605.06527 (2026)
충돌	Temporal Fact Conflicts (DYNAMICQA/MULAN)	arXiv 2603.15892 (2026)
회수	Aingram recall@k (로컬)	dev.to, LongMemEval recall_any@10=0.955
회수	HopRAG (multi-hop traversal)	arXiv 2502.12442
회수	Hindsight / TEMPR	arXiv 2512.12818 (2026)

영역	연구 / 시스템	식별자 / 비고
벤치마크	MemoryAgentBench (FactConsolidation)	arXiv 2507.05257 (2025/26)
벤치마크	LongMemEval	Wu et al., ICLR 2025
벤치마크	LoCoMo / BEAM	장기 대화 / 1M·10M 토큰 스케일
서베이	Memory for Autonomous LLM Agents	arXiv 2603.07670 (2026)
서베이	From Human Memory to AI Memory	arXiv 2504.15965 (2025)

주의: 본 설계의 두 핵심 충돌 논문(결정론적 Freshness, Engram)과 일부 2026년 논문은 작성 시점 기준 매우 최신(수 주 전 발표)이라 구현 성숙도는 직접 검증이 필요하다. 두 Engram 논문 모두 오픈소스·로컬 우선 설계이므로 참고·차용에 적합하다.